



Livrable Livrable

Bloc développement d'applications distribuées

Groupe 1 :

- BOUZJALLIKHT Yanis
- HASSANI Rayane
- FARDELLA Timothé
- ROCHE Gabriel
- MARIANI Mattéo

Table des matières

1. Introduction.....	4
1.1 Contexte et genèse du projet	4
1.2 Problématique et besoins identifiés	4
1.3 Périmètre	4
2. Objectifs et attentes du projet	6
2.1 Objectifs fonctionnels	6
2.2 Objectifs techniques	6
2.3 Critères de succès et indicateurs de performance	7
3. Architecture du système	8
3.1 Vue d'ensemble de l'architecture microservices	8
3.2 Description des services et leurs responsabilités	8
3.3 Communication inter-services et sécurité (mTLS / PKI)	11
3.4 Infrastructure et environnement de déploiement (Docker, Nginx, Redis)	15
4. Ressources et organisation du projet.....	20
4.1 Stack technique et outils mobilisés.....	20
4.2 Organisation du dépôt (monorepo Turborepo).....	21
4.3 Répartition des responsabilités	22
Structure et rôles au sein de l'équipe	22
Répartition opérationnelle des tâches	22
Méthodologie de gestion des sources et flux de travail (Workflow)	23
5. Méthodologie et étapes de développement	24
5.1 Approche adoptée	24
5.2 Phases du projet (conception → développement → tests → déploiement)	24
5.3 Gestion de version et workflow Git	25
6. Interfaces utilisateur	26
6.1 Charte graphique et principes UX.....	26
6.2 Maquettes ou wireframes des écrans principaux.....	30
6.3 Navigation et parcours utilisateur	31
7. Fonctionnalités principales	34

7.1 Authentification et gestion des utilisateurs	34
7.2 Gestion des publications (posts)	35
7.3 Système de notifications.....	36
7.4 Système de messagerie privée.....	37
7.5 Passerelle API et routage (Gateway Nginx)	38
8. Perspectives et axes d'amélioration	40
8.1 Évolutions fonctionnelles envisagées	40
8.2 Améliorations techniques (scalabilité, monitoring, CI/CD...)	40
8.3 Pistes d'optimisation de l'expérience utilisateur (UX).....	41
Conclusion	42

1. Introduction

1.1 Contexte et genèse du projet

Le projet Breezy s'aligne sur la tendance actuelle des réseaux sociaux, dominée par des grandes plateformes telles que Twitter/X. Néanmoins, l'utilisation de ces services exige fréquemment des infrastructures complexes et une large capacité de transmission de données, engendrant ainsi une division numérique pour les utilisateurs possédant des équipements ou des connexions restreints. Breezy est né du désir de créer une autre option : un réseau de microblogging qui soit à la fois léger, rapide et ajusté pour des contextes à ressources limitées. L'idée est d'offrir une expérience utilisateur sans accroc, rapide et focalisée sur le primordial : la diffusion de messages brefs et l'échange immédiat entre les membres de la communauté.

1.2 Problématique et besoins identifiés

Le principal défi de ce projet réside dans l'alliance de la richesse fonctionnelle d'un réseau social contemporain avec des contraintes strictes de performance et d'interopérabilité. On distingue trois axes majeurs autour desquels s'organisent les besoins identifiés :

- **Accessibilité et aisance** : Élaborer une interface utilisateur adaptative en suivant une stratégie mobile-first, assurant une présentation parfaite sur tous les formats d'écran.
- **Sécurité et confidentialité** : Il est crucial de garantir une authentification solide pour préserver les données sensibles, étant donné la nature prolifique des échanges sur la plateforme (messages privés, publications, abonnements).
- **Robustesse et évolutivité** : Établir une architecture capable de s'adapter à l'augmentation de la charge, tout en facilitant le déploiement technique et l'entretien.

1.3 Périmètre

Le champ fonctionnel et technique de Breezy est strictement déterminé afin de satisfaire les objectifs d'agilité du projet :

- **Etendue fonctionnelle et services** : L'application est construite sur une base de micro-services, comportant cinq services différents et reliés entre eux, chacun gérant une obligation business distincte (par exemple, la gestion des utilisateurs, les publications, le système de notifications ou la messagerie privée).

- **Configuration et déploiement ciblé** : Pour assurer la portabilité, la fiabilité et une mise en production rapide, tout le projet repose sur l'utilisation de Docker pour la conteneurisation. Cette configuration va permettre de séparer chaque élément et de faciliter la gestion des dépendances, de l'étape de développement jusqu'à la phase de production.
- **Portée fonctionnelle** : Au cœur de l'application se trouvent des fonctionnalités indispensables (création de compte avec vérification, authentification protégée par JWT, flux d'actualités chronologique, mentions j'aime, commentaires, abonnements et profils des utilisateurs). De plus, un assortiment de fonctionnalités additionnelles a été prévu pour améliorer l'expérience de l'utilisateur, notamment : l'implémentation de balises de recherche, un mécanisme de notifications, un service de messagerie privée sécurisée, la gestion de contenus multimédias (photos et vidéos), ainsi qu'un instrument de modération autorisant la dénonciation et l'exclusion. L'usage de ces fonctionnalités est strictement gouverné par une grille de permissions basée sur quatre profils différents : Visiteur, Utilisateur, Modérateur et Administrateur.

2. Objectifs et attentes du projet

2.1 Objectifs fonctionnels

WeTalk est un réseau social interactif dont la mission est de faciliter la communication entre ses membres de manière sécurisée et instantanée.

L'expérience utilisateur repose sur un socle de fonctionnalités clés : la gestion autonome des comptes, la maîtrise complète de ses publications, la personnalisation de son profil et la réception de notifications en temps réel.

Ces éléments constituent l'essence même de la plateforme, offrant ainsi un service performant et centré sur les besoins de ses utilisateurs.

2.2 Objectifs techniques

Nos équipes poursuivent des objectifs techniques concrets pour obtenir une architecture fiable et efficace.

Le premier objectif fixé concerne la modularité de notre projet, nous avons conçu notre architecture sous la forme de micro-services indépendants pour assurer une meilleure scalabilité pour les prochaines mises à jour.

Au cœur de notre architecture, nous visons également d'assurer une communication sécurisée et fluide entre les services présents pour garantir au mieux une application fiable et fonctionnelle.

En parallèle de la conception de l'architecture avec des micro-services, nous avons également pour but d'optimiser la portabilité de notre application en conteneurisant l'ensemble du système. Cette démarche assure une stricte similarité entre les environnements de développement et de production.

Pour finir, nous avons fait le choix de centraliser la configuration TypeScript partagée via un package dédié (typescript-config). Cet objectif nous permettra d'assurer la maintenabilité de notre code.

2.3 Critères de succès et indicateurs de performance

Nous considérerons le projet comme abouti lorsque les conditions suivantes seront réunies :

- L'ensemble des services démarre correctement et communique sans erreur à travers Docker (+ communication en HTTP)
- Aucun service interne n'est directement accessible depuis l'extérieur du réseau Docker (obligation de passer par l'API Gateway)
- Les fonctionnalités principales (authentification, publication, notification, les messages etc.) sont opérationnelles, un utilisateur peut utiliser l'application sans soucis.
- Le projet est favorable à la scalabilité, la maintenabilité, la disponibilité et enfin le projet est performant.

3. Architecture du système

3.1 Vue d'ensemble de l'architecture microservices

3.1.1 Pourquoi une architecture microservices

Pour WeTalk, on a choisi de découper l'application en plusieurs services indépendants plutôt que de tout mettre dans un seul projet (Monolithique). Ce choix s'explique principalement par la nature de la plateforme : la gestion des utilisateurs, la publication de contenu, la messagerie et les notifications sont des fonctionnalités suffisamment distinctes pour vivre séparément. Chaque service peut ainsi évoluer, être redémarré ou mis à jour sans impacter les autres.

Cela a aussi une conséquence sur les bases de données : chaque service possède la sienne adaptée à ses besoins. On ne partage pas une seule base commune entre tous les services — c'est l'un des principes fondamentaux de cette architecture.

3.2 Description des services et leurs responsabilités

« auth-service »

Le service d'authentification est le point d'entrée pour toute création de compte et toute connexion. Il est le seul à manipuler les mots de passe et à émettre des jetons JWT.

À la connexion, il génère deux jetons : un jeton d'accès de courte durée et un jeton de renouvellement de longue durée. Ces deux jetons sont déposés dans des cookies HttpOnly, invisibles au JavaScript du navigateur, ce qui protège contre les attaques XSS. Le jeton de renouvellement est enregistré dans Redis avec un identifiant unique, ce qui permet de le révoquer immédiatement en cas de bannissement.

Il prend également en charge les limites de tentatives de connexion via un système de rate limiting pour contrer les attaques par force brute.

Technologies : Node.js, Express, PostgreSQL, Redis, bcryptjs, jsonwebtoken

« user-service »

Le service utilisateurs centralise tout ce qui touche aux profils. Il gère la mise à jour des informations personnelles (nom affiché, biographie, avatar, bannière), les relations de suivi entre utilisateurs, ainsi que les actions de modération comme le bannissement et la suspension de comptes.

C'est aussi lui qui est interrogé par les autres services pour enrichir leurs données — le service des publications l'appelle par exemple pour récupérer le nom et la photo de profil de l'auteur d'un post.

Technologies : Node.js, Express, PostgreSQL, Redis

« post-service »

Le service des publications gère l'ensemble du cycle de vie du contenu : création, récupération, suppression, likes et commentaires. Il orchestre aussi la chaîne média — quand un utilisateur publie une photo ou une vidéo, il relaie le fichier vers le media-service et stocke l'URL retournée dans le document MongoDB.

Il intègre également le système de signalement et les fonctions de modération : les modérateurs et administrateurs peuvent consulter les signalements en attente, ignorer un rapport ou supprimer le contenu concerné.

Lors de la lecture des publications, les profils des auteurs bannis sont automatiquement masqués — le contenu n'est pas supprimé en base, mais filtré à l'affichage, ce qui rend l'opération réversible en cas d'unban.

Technologies : Node.js, Express, MongoDB, Mongoose

« message-service »

Le service de messagerie gère les conversations privées entre utilisateurs. Il s'appuie sur deux canaux complémentaires : une API REST pour envoyer un message et récupérer l'historique d'une conversation, et une connexion Socket.io pour la réception en temps réel.

Quand un message est envoyé via l'API, le service le sauvegarde en base puis émet immédiatement un événement Socket.io vers la room privée du destinataire. Si ce dernier est connecté, il reçoit le message instantanément sans avoir besoin de rafraîchir la page. Avant tout envoi, le service vérifie auprès du service utilisateurs que ni l'expéditeur ni le destinataire ne sont bannis ou suspendus.

Technologies : Node.js, Express, MongoDB, Socket.io

« notification-service »

Le service de notifications crée et distribue les alertes déclenchées par les actions des autres utilisateurs — un like, un commentaire, ou un nouveau follower. Il est appelé

en interne par le service des publications via un endpoint réservé aux échanges inter-services, bloqué publiquement au niveau de la passerelle Nginx.

La distribution en temps réel fonctionne également via Socket.io : chaque utilisateur connecté rejoint automatiquement une room privée identifiée par son UUID, et reçoit ses notifications sans délai.

Technologies : Node.js, Express, MongoDB, Socket.io

« media-service »

Le service média a une responsabilité unique : stocker et servir les fichiers uploadés par les utilisateurs. Il reçoit les fichiers envoyés par le service des publications, les enregistre sur un volume Docker persistant et retourne une URL d'accès. Il vérifie également que la taille et le format du fichier respectent les limites définies avant de l'accepter.

Les fichiers sont servis directement via la passerelle Nginx sans passer par la logique applicative, ce qui allège la charge du service.

Technologies : Node.js, Express, Multer

« Application web (Next.js) »

L'application web est le seul élément de la plateforme directement visible par l'utilisateur. Elle est construite avec Next.js en utilisant l'App Router, ce qui permet de mélanger rendu côté serveur et composants interactifs côté client selon les besoins de chaque page.

Les pages dont le contenu doit être indexé ou chargé rapidement (profil, publications) sont rendues côté serveur. Les fonctionnalités interactives comme la messagerie, les likes ou les formulaires utilisent des composants client. Toutes les requêtes vers l'API passent par un wrapper commun apiFetch, qui gère automatiquement le renouvellement de session en cas d'expiration du jeton d'accès.

L'interface est entièrement traduite en français et en anglais via next-intl.

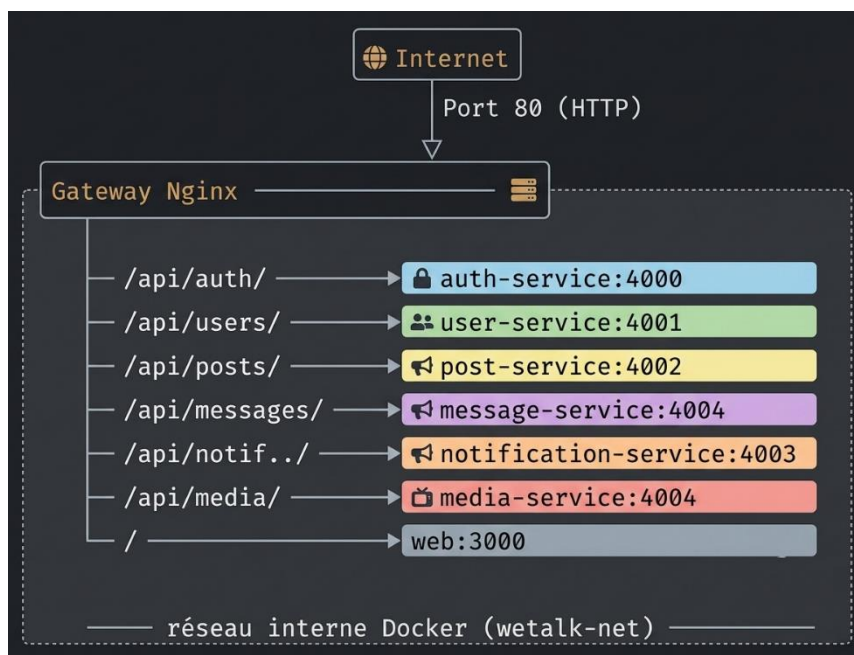
Technologies : Next.js 15, React, Tailwind CSS, Radix UI, Socket.io client, next-intl

3.3 Communication inter-services et sécurité (mTLS / PKI)

3.3.1 Communication inter-service

Au sein de WeTalk, l'ensemble des services tournent dans le même réseau Docker privé que l'on a nommé « wetalk-net ». Concrètement ils peuvent se contacter entre eux, mais restent inaccessibles depuis l'extérieur. Le seul point d'entrée public est la passerelle Nginx sur le port 80, qui redirige chaque requête vers le bon service.

Le schéma ci-dessous résume l'organisation de l'architecture microservices :



Un point important : dans le fichier docker-compose.yml, on utilise le mot-clé `expose` plutôt que `ports` pour les services. La différence est simple « `— expose` » rend un port accessible uniquement aux autres conteneurs du même réseau alors que « `ports` » l'ouvrirait directement sur la machine hôte. C'est une première protection réseau simple mais efficace.

Pour les échanges entre services, on utilise du HTTP classique sur le réseau interne Docker. On a choisi de ne pas chiffrer ce trafic car le réseau Docker est déjà isolé — on reviendra sur les limites de ce choix à la fin de cette partie.

3.3.2 Timeouts et gestion des pannes

Chaque appel entre services dispose d'un délai maximum pour éviter qu'un service lent ne bloque tout le reste.

Appel	Délai maximum
Vérification du statut d'un utilisateur	3 secondes
Récupération du profil d'un auteur	3 secondes
Récupération du profil d'un auteur	15 secondes
Création d'une notification interne	3 secondes

Les appels critiques sont distingués des appels secondaires. Par exemple, si l'envoi d'une notification échoue, la publication est quand même créée « — l'erreur » est simplement enregistrée dans les journaux. En revanche, si la vérification du statut banni d'un utilisateur échoue avant l'envoi d'un message, l'opération est bloquée. C'est ce qu'on appelle respectivement le mode dégradé et le mode bloquant.

3.3.3 Authentification entre services

Le problème à résoudre était le suivant : lorsque le service des publications appelle le service utilisateurs pour récupérer le profil d'un auteur, comment ce dernier sait-il que la requête est légitime ?

On a opté pour une **vérification locale des jetons JWT**. Chaque service connaît le secret « JWT_ACCESS_SECRET » et peut contrôler lui-même la signature d'un jeton sans avoir à interroger l'auth-service. Cela évite un appel réseau supplémentaire à chaque requête et supprime une dépendance potentiellement bloquante.

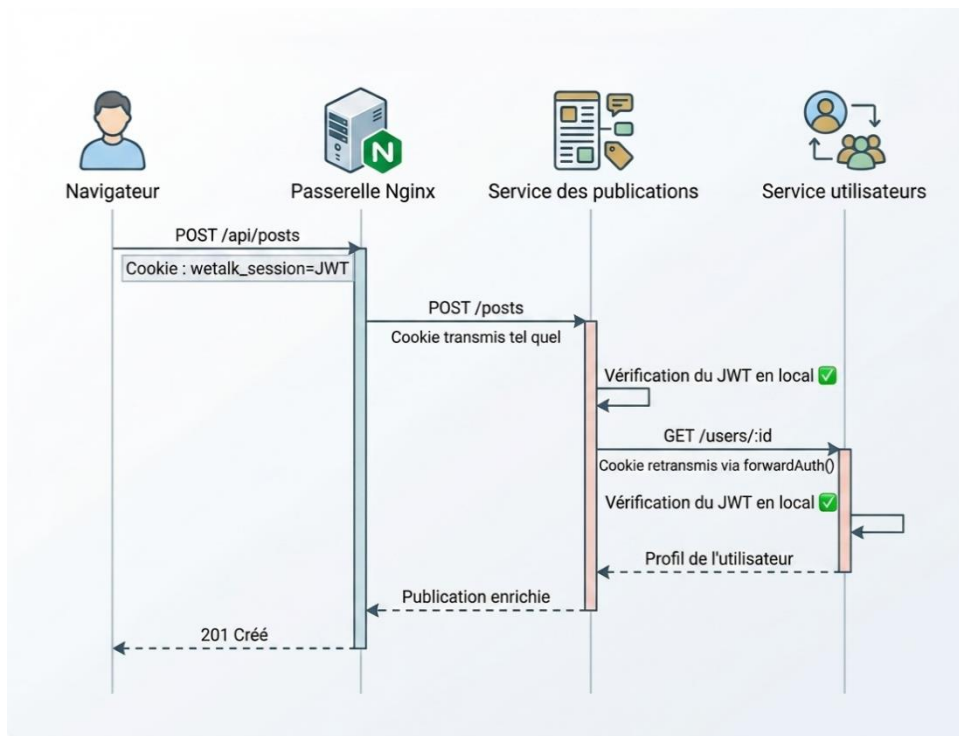
En pratique, quand une requête arrive du navigateur avec le cookie « wetalk_session », le premier service qui la reçoit relaie ce cookie à tous les services qu'il contacte ensuite grâce à la fonction « forwardAuth() » :

```

1  function forwardAuth(req: Request): Record<string, string> {
2    const headers: Record<string, string> = {};
3    if (req.headers.authorization)
4      headers.Authorization = req.headers.authorization;
5    if (req.headers.cookie)
6      headers.cookie = req.headers.cookie;
7    return headers;
8  }

```

Chaque service destinataire vérifie ensuite le jeton de son côté avec son propre middleware. Le schéma illustre ce flux :



3.3.4 Révocation des jetons

La vérification locale a un défaut connu : si un jeton est valable 15 minutes et qu'un utilisateur est banni à la deuxième minute, son jeton reste techniquement valide jusqu'à la quinzième. Pour corriger ça, on a mis en place deux mécanismes dans Redis.

La liste blanche des jetons de renouvellement. À chaque connexion, un identifiant unique « jti » est généré pour le jeton de renouvellement et stocké dans le

cache Redis. Lors d'une demande de renouvellement, on vérifie que ce « jti » est toujours présent. Si l'utilisateur a été banni entre-temps, le « jti » est retiré de Redis et la session se termine proprement. Ce mécanisme protège aussi contre la réutilisation d'un jeton volé puisque chaque renouvellement génère un nouveau « jti » et invalide l'ancien.

La liste noire des comptes bannis. Lors d'un bannissement, l'identifiant de l'utilisateur est ajouté dans une liste noire Redis avec une durée de vie égale à celle du jeton d'accès. Chaque middleware d'authentification consulte cette liste après avoir validé la signature. Un utilisateur banni est donc bloqué dès sa prochaine requête sans attendre l'expiration naturelle de son jeton.

3.3.5 Blocage des routes internes

Certains points d'accès sont réservés aux échanges entre services et ne doivent jamais être joignables depuis Internet. C'est le cas de « POST /notifications/interna » utilisé par le service des publications pour créer des notifications. Une règle dans la configuration Nginx bloque cette route en amont :



```
1 location ~ ^/api/notifications/internal(?:/|$) {  
2     return 404;  
3 }
```

Les services qui ont besoin d'appeler cette route le font directement via le réseau Docker interne sans passer par la passerelle.

3.4 Infrastructure et environnement de déploiement (Docker, Nginx, Redis)

3.4.1 Conteneurisation avec Docker

Toute l'infrastructure de WeTalk est conteneurisée avec Docker. Chaque service dispose de son propre conteneur construit à partir d'un « Dockerfile » dédié. L'orchestration de l'ensemble est gérée par un fichier « docker-compose.yml » à la racine du projet qui définit les services, leurs dépendances, le réseau commun et les volumes persistants.

Cette approche présente plusieurs avantages pour notre projet. D'abord, chaque développeur peut lancer l'intégralité de la plateforme avec une seule commande « docker compose up » sans avoir à installer manuellement Node.js, PostgreSQL, MongoDB ou Redis. Ensuite, les environnements de développement et de production sont identiques ce qui évite les problèmes du type « ça marche sur ma machine ».

3.4.2 Organisation des conteneurs

Le fichier « docker-compose.yml » définit onze conteneurs au total que l'on peut regrouper en trois catégories :

Les bases de données

Conteneur	Image	Rôle
wetalk-postgres	postgres:16-alpine	Données structurées — comptes utilisateurs, sessions
wetalk-mongo	mongo:7	Données non structurées — publications, messages, notifications
wetalk-redis	redis:7-alpine	Cache et révocation des jetons

Les services applicatifs

Conteneur	Port interne	Dépend de
wetalk-auth-service	4000	PostgreSQL, Redis
wetalk-user-service	4001	PostgreSQL, Redis, media-service
wetalk-post-service	4002	MongoDB, user-service, media-service
wetalk-message-service	4004	MongoDB, Redis, user-service
wetalk-notification-service	4003	MongoDB, user-service
wetalk-media-service	4004	--

La couche présentation et routage

Conteneur	Port exposé	Rôle
wetalk-web	3000 (interne)	Application Next.js
wetalk-gateway	80 (public)	Passerelle Nginx

3.4.3 Réseau et Isolation

Tous les conteneurs sont connectés au même réseau Docker de type bridge. Ce réseau est entièrement privé, aucun service applicatif n'expose de port sur la machine hôte. Seule la passerelle publie le port 80.

La résolution des noms se fait automatiquement par Docker : le service post-service peut appeler `http://user-service:4001` directement, Docker s'occupant de traduire ce nom en adresse IP interne. C'est pour cette raison qu'on retrouve ces noms dans les variables d'environnement de chaque service :

```

1 # Extrait docker-compose.yml – post-service
2 environment:
3   USER_SERVICE_URL: http://user-service:4001
4   MEDIA_SERVICE_URL: http://media-service:4004
5   NOTIFICATION_SERVICE_URL: http://notification-service:4003

```


3.4.4 Ordre de démarrage et vérifications de santé

Certains services dépendent d'autres pour fonctionner — le service des publications ne peut pas démarrer si MongoDB n'est pas prêt. Docker Compose gère cet ordre grâce aux directives « `depends_on` » combinées à des « `healthchecks` ».

Chaque conteneur définit une vérification de santé : une commande exécutée périodiquement qui permet à Docker de savoir si le service est opérationnel. Par exemple pour le service d'authentification :

```
1 healthcheck:
2   test: ["CMD", "node", "-e",
3     "require('http').get('http://localhost:4000/health',
4     r=>process.exit(r.statusCode===200?0:1))
5     .on('error',()=>process.exit(1))"]
6   interval: 10s
7   timeout: 3s
8   retries: 5
9   start_period: 20s
```

Cette commande interroge le endpoint « `/health` » du service toutes les 10 secondes. Si elle échoue cinq fois de suite, Docker considère le conteneur comme défaillant. Les services qui en dépendent attendent que ce healthcheck soit passé avant de démarrer grâce à « `condition: service_healthy` » :

```
1 depends_on:
2   postgres:
3     condition: service_healthy
4   redis:
5     condition: service_healthy
```

3.4.5 La passerelle Nginx

Nginx joue le rôle de passerelle unique entre Internet et les services internes. Sa configuration repose sur un système de blocs « `location` » qui associent un préfixe d'URL à un service cible.

Quelques points notables de la configuration :

Routage par préfixe

Chaque service est joignable via un préfixe « `/api/nom-du-service/` ». Nginx retire ce préfixe avant de transmettre la requête ce qui fait que le service reçoit une URL propre :

```
1 location /api/auth/ {  
2     proxy_pass http://auth_service/auth/;  
3 }
```

Cas particulier des publications

La route « /api/posts » nécessite deux règles distinctes — une pour la collection sans slash final (liste et création) et une pour les routes avec slash (accès individuel) :

```
1 location = /api/posts {  
2     proxy_pass http://post_service/posts;  
3 }  
4 location /api/posts/ {  
5     proxy_pass http://post_service/posts/;  
6 }
```

Support des WebSockets

La messagerie et les notifications utilisent Socket.io qui nécessite une connexion WebSocket persistante. Nginx gère cela via une directive de mise à niveau du protocole :

```
1 map $http_upgrade $connection_upgrade {  
2     default upgrade;  
3     ''      close;  
4 }
```

3.4.6 Le rôle de Redis

Redis est utilisé à deux endroits distincts dans WeTalk, avec des responsabilités bien séparées.

Gestion des sessions dans l'auth-service

Redis stocke la liste blanche des jetons de renouvellement (jti) et la liste noire des comptes bannis. Ces deux structures ont une durée de vie limitée (TTL) alignée sur celle des jetons correspondants, ce qui garantit un nettoyage automatique sans intervention manuelle.

Vérification du bannissement dans le user-service

Quand un modérateur banne un utilisateur, l'information est écrite dans Redis en plus de la base de données PostgreSQL. Cela permet aux autres services de vérifier très rapidement si un compte est banni sans faire de requête SQL à chaque appel entrant.

Redis est volontairement invisible de l'extérieur — son port 6379 n'est exposé que sur le réseau interne Docker, et aucune route Nginx ne le cible directement.

4. Ressources et organisation du projet

4.1 Stack technique et outils mobilisés

L'application WeTalk s'appuie sur une architecture en micro-services autonomes qui communiquent via des API REST. Ce choix architectural répond aux exigences de scalabilité d'un réseau social, permettant de faire monter en charge un composant spécifique de manière indépendante, sans nécessiter un redéploiement global de l'infrastructure. À titre d'exemple, une augmentation massive du volume des publications implique uniquement de scaler horizontalement le service dédié, ici le service des post. De plus, l'isolation des microservices garantit une meilleure résilience globale en évitant les pannes à la chaîne.

Le frontend est développé avec le framework Next.js, basé sur la bibliothèque JavaScript React et entièrement en TypeScript. Le framework Next.js a été sélectionné pour sa gestion native du routage, du rendu et de l'organisation des pages. L'utilisation de l'App Router, associée à une structure de répertoires localisés (app/[locale]/), assure la gestion native du multilinguisme (français et anglais).

Le stack technologique de l'interface intègre également :

- Tailwind CSS : pour le stylisme via des classes utilitaires (CSS-in-JS / Utility-first)
- Radix UI et Lucide-react : pour des composants d'interface pré-fait et une bibliothèque d'icônes à utiliser
- Next-intl et Next-themes : pour la gestion de l'internationalisation et des thèmes d'affichage (ambre / rose / sombre)
- Socket.io-client : pour la communication en temps réel (notifications et messagerie instantanée).

Concernant le Backend, chaque microservice backend est structuré de manière homogène sous la forme d'une API REST développée avec Express et TypeScript. Les technologies suivantes ont été implémentées :

- Authentification : sécurisée par jetons JWT (via Access Token et Refresh Token)
- Express-rate-limit : pour la limitation des requêtes afin de prévenir les abus
- Helmet et CORS : pour le durcissement de la sécurité des en-têtes HTTP et le contrôle des origines autorisées ;
- Zod : pour la validation et le typage strict des données d'entrée avant le traitement métier ;
- Multer : pour la gestion et le téléversement des fichiers médias.

Conformément aux bonnes pratiques de l'architecture en microservices, chaque service possède sa propre base de données isolée, interdisant tout accès direct d'un service tiers à une base de données qui ne lui appartient pas. Les technologies de persistance sont adaptées selon la nature des données :

- PostgreSQL (avec l'ORM Sequelize) : pour les données relationnelles structurées (gestion des user)
- MongoDB (avec l'ODM Mongoose) : pour le stockage documentaire de données semi-structurées (gestion des posts, notifications, messages)
- Redis : utilisé pour la gestion du cache et du cycle de vie des JWT.

L'infrastructure s'organise autour d'une API Gateway Nginx, qui est le point d'entrée unique de l'application. Elle assure le routage des requêtes vers les services concernés. L'ensemble des services est conteneurisé grâce à Docker, permettant le déploiement et l'exécution de l'intégralité du projet via une commande unique.

Enfin, l'orchestration du monorepo (builds, pipelines et environnement de développement) est centralisée et optimisée par Turborepo.

Cette architecture est complétée par un pipeline d'intégration continue (CI) et de déploiement continu (CD) automatisé via GitHub Actions. Le workflow de CI valide chaque modification de code en moins de 2 minutes 30 en exécutant en parallèle des analyses de sécurité statiques (détection de fuites de secrets avec Gitleaks, ainsi qu'une recherche de vulnérabilités applicatives et d'infrastructure via CodeQL et Trivy), les étapes de contrôle qualité (linting, vérification du typage strict et tests unitaires) ainsi que la compilation globale. Une fois ces validations validées avec succès, le pipeline de CD prend le relais pour automatiser le déploiement des conteneurs applicatifs sur un serveur privé (VPS) orchestré avec dockploy afin de rendre l'application accessible sur Internet. Enfin, une notification est instantanément envoyée sur Discord pour informer l'équipe du succès ou de l'échec de l'ensemble de la procédure.

4.2 Organisation du dépôt (monorepo Turborepo)

L'intégralité de la solution logicielle est centralisée au sein d'un dépôt unique (monorepo), regroupant l'application frontend, la passerelle d'API (gateway), l'ensemble des micro-services ainsi que les modules de code partagés.

La structure de l'arborescence s'organise autour d'un seul répertoire principal :

- **apps/** : regroupe l'ensemble des applications et services exécutables destinés à être déployés de manière indépendante. Chaque sous-répertoire intègre son

propre fichier de configuration `Dockerfile`, garantissant l'isolation de son processus de conteneurisation.

La gestion et la maintenance de ce monorepo sont orchestrées par l'outil Turborepo. Ce gestionnaire de tâches optimise les flux de travail grâce à un système de cache avancé : lors de l'exécution d'une commande globale de compilation (build), seules les portions de code modifiées sont recompilées, évitant ainsi les traitements redondants. De plus, les tâches indépendantes sont exécutées en parallèle, ce qui réduit considérablement les temps d'intégration continue. Enfin, la centralisation des sources au sein d'un même historique de versions simplifie le cycle de livraison, permettant de lier les modifications d'interface et de logique métier (frontend et backend) au sein d'une unique requête de fusion (*Pull Request*).

4.3 Répartition des responsabilités

Structure et rôles au sein de l'équipe

Afin d'assurer le développement et le déploiement de la solution, les rôles au sein de l'équipe projet ont été définis de la manière suivante :

- Chef de projet : Timothé
- Ingénieurs Backend : Rayane et Gabriel
- Ingénieur DevOps : Yanis
- Ingénieur Frontend : Mattéo

Répartition opérationnelle des tâches

La charge de travail a été segmentée par pôles de compétences techniques, favorisant une collaboration transverse :

- **Développement Frontend (web)** : L'implémentation de l'interface utilisateur et son interconnexion avec le backend a été réalisé par Mattéo, Rayane et Gabriel.
- **Développement Backend** : La conception, l'architecture et le développement des différents microservices ont été répartis entre Gabriel, Rayane et Timothé.
- **Infrastructure et DevOps** : La configuration de la conteneurisation (Docker), la mise en place de la passerelle de routage (Gateway), l'écriture des scripts d'automatisation (Provisioner) ainsi que le déploiement continu via l'outil Dokploy ont fait l'objet d'un travail collaboratif entre Rayane, Yanis et Timothé.
- **Pilotage et Gestion de Projet** : La planification, le suivi et la coordination générale ont été supervisés par Timothé, chef de projet, en concertation avec l'ensemble des membres de l'équipe.

Méthodologie de gestion des sources et flux de travail (Workflow)

Sur le plan méthodologique, le cycle de développement s'appuie sur les outils Git et GitHub, en suivant un flux de travail rigoureux par fonctionnalités :

- **Isolation des développements** : Chaque nouvelle fonctionnalité ou correction d'anomalie fait l'objet d'une branche dédiée et d'un ou plusieurs commits, nommées selon les bonnes pratiques de nommage.
- **Contrôle qualité et revue de code** : L'intégration de tout code au sein de la branche principale (`main`) requiert obligatoirement l'ouverture d'une demande de fusion (*Pull Request*). Une relecture croisée par les pairs est systématiquement effectuée avant validation, ainsi que l'approbation du pipeline de CI.
- **Intégrité de la branche principale** : Les push directes sur la branche `main` sont strictement interdites par des règles de protection de branche, garantissant ainsi la stabilité permanente du code en production.

5. Méthodologie et étapes de développement

5.1 Approche adoptée

Le développement de WeTalk a suivi une approche itérative et incrémentale. L'idée était simple : poser les fondations de l'architecture en premier, puis ajouter les services un à un, et enfin affiner la configuration jusqu'à obtenir un environnement stable et fonctionnel.

Ce choix s'est avéré pertinent. Plusieurs problèmes ont été détectés et corrigés au fil du développement sans bloquer l'avancement global.

Chaque membre du groupe avait son rôle et suivait une trame bien définie par le chef de projet.

5.2 Phases du projet (conception → développement → tests → déploiement)

Phase 1 : Conception de l'architecture

Avant de coder, on a défini la structure globale du système. Cela incluait l'identification des services nécessaires, le choix des bases de données associées à chacun (PostgreSQL pour l'authentification et les utilisateurs, MongoDB Atlas pour les publications, Redis pour le cache), et enfin la conception du schéma réseau en utilisant Docker avec Nginx comme unique point d'entrée externe (API Gateway)

Phase 2 — Mise en place du monorepo

La structure Turborepo avec npm workspaces a été initialisée en premier. On a créé le package partagé (typescript-config) pour centraliser la configuration TypeScript commune à tous les services. Cette étape a posé les bases de l'ensemble du projet.

Phase 3 — Développement des services

Chaque service a été développé de manière indépendante, en suivant une trame commune : un point d'entrée `src/index.ts`, une connexion à sa BDD, et l'exposition de ses routes. Les Dockerfiles suivent tous une construction en trois étapes (base, builder, et runner) pour optimiser le cache des couches + limiter la taille des images.

Phase 4 — Intégration Docker Compose

L'ensemble des services a été assemblé dans un `docker-compose.yml` unique. Cette phase a couvert la configuration des healthchecks sur PostgreSQL, la gestion des dépendances entre services via `depends_on` avec condition `service_healthy`, et la mise en place du réseau bridge `wetalk-net`.

On a décidé d'utiliser `expose` plutôt que `ports` pour tous les services internes car `expose` rend un port accessible uniquement aux autres conteneurs présents dans le réseau, si on avait utilisé « `ports` » ça aurait ouvert des accès sur la machine hôte, sauf qu'on

souhaite que Nginx uniquement soit exposé à l'extérieur et agisse en tant qu'API Gateway (ports 80 & 443)

Phase 6 — Tests et corrections

Des tests fonctionnels ont été effectués pour vérifier le démarrage de l'ensemble des services, la cohérence du routage Nginx, et l'absence d'erreurs de résolution TypeScript en local.

Lorsque des erreurs de résolution apparaissaient, la première action était de relancer npm install à la racine du monorepo pour résoudre les dépendances cross-workspace et de redémarrer le serveur TypeScript.

Dans la majorité des cas, cela suffisait avant d'envisager un problème de configuration Docker.

5.3 Gestion de version et workflow Git

Le projet est versionné avec Git autour du monorepo. Chaque service dispose de son propre répertoire dans apps/.

Les fichiers sensibles, variables d'environnement etc sont exclus du versionnement via un .gitignore. Les fichiers .env ne sont jamais commités pour des questions de sécurité. Un fichier .env.example documente les variables attendues pour chaque service.

6. Interfaces utilisateur

6.1 Charte graphique et principes UX

6.1.1 Identité visuelle

Le parti pris de WeTalk est celui de la chaleur. Là où la majorité des réseaux sociaux misent sur le blanc clinique ou le noir total, on a choisi une palette de tons terreux et dorés qui rappellent le papier kraft, la lumière du soir, l'ambre. L'idée derrière ça, c'est que l'interface elle-même doit donner envie de rester pas seulement le contenu.

Le slogan "*Le côté chaleureux du social*" n'est pas qu'un argument marketing : il a guidé chaque décision de design, de la couleur de fond jusqu'aux arrondis des cartes.

6.1.2 Palette de couleurs

Le système de couleurs est entièrement défini en variables CSS, ce qui permet de changer de thème à la volée sans recharger la page. Trois thèmes sont proposés : Ambre (par défaut), Rose et Sombre.

Rôle	Variable CSS	Valeur hex	Usage
Accent principal	--gold	#ba7517	Boutons, liens actifs, focus
Accent lumineux	--gold-bright	#ef9f27	Hover, dégradés
Fond global	--canvas	#f0eae0	Arrière-plan de la page
Fond carte	--card	#ffffff	Posts, modales, panneaux
Fond doux	--cream	#faf7f2	Inputs, zones de saisie
Texte principal	--brown	#412402	Corps de texte
Texte secondaire	--brown-sec	#854f0b	Métadonnées, placeholders
Placeholder	--placeholder	#b69b78	Texte indicatif dans les champs
Bordure	--border	#e8dfd4	Séparateurs, contours
Alerte / Danger	--live (fixe)	#dc2626	Erreurs, bandeaux bannissement
Badge vérifié	--blue (fixe)	#1d9bf6	Compte certifié
Surface sombre	--dark (fixe)	#1a1410	Overlay, fond téléphone démo

6.1.3 Typographie

Deux familles de polices, toutes les deux chargées via « next/font » pour éviter le flash de police au chargement.

Rôle	Variable CSS	Valeur hex	Usage
Accent principal	--gold	#ba7517	Boutons, liens actifs, focus
Accent lumineux	--gold-bright	#ef9f27	Hover, dégradés
Fond global	--canvas	#f0eae0	Arrière-plan de la page
Fond carte	--card	#ffffff	Posts, modales, panneaux

Fond doux	--cream	#faf7f2	Inputs, zones de saisie
Texte principal	--brown	#412402	Corps de texte
Texte secondaire	--brown-sec	#854f0b	Métadonnées, placeholders
Placeholder	--placeholder	#b69b78	Texte indicatif dans les champs
Bordure	--border	#e8dfd4	Séparateurs, contours
Alerte / Danger	--live (fixe)	#dc2626	Erreurs, bandeaux bannissement
Badge vérifié	--blue (fixe)	#1d9bf6	Compte certifié
Surface sombre	--dark (fixe)	#1a1410	Overlay, fond téléphone démo

6.1.4 Formes et ombres

Les rayons d'arrondi sont définis globalement pour garantir la cohérence sur tous les composants.

Token	Valeur	Application
radius-field	13px	Champs de saisie, badges, tags
radius-card	24px	Cartes de post, modales, panneaux flottants

Les ombres sont teintées dans les mêmes tons chauds que la palette mais jamais en gris neutre.

Token ombre	Valeur CSS	Usage
shadow-gold	0 8px 20px rgba(186,117,23,0.32)	Bouton CTA au hover
shadow-soft	0 2px 10px rgba(65,36,2,0.06)	Élévation subtile des cartes
shadow-card	0 24px 60px rgba(65,36,2,0.10)	Modales et panneaux flottants

6.1.5 Animations et micro-interactions

Toutes les animations sont définies en CSS et respectent la préférence système “prefers-reduced-motion” si l'utilisateur a activé la réduction de mouvement dans son OS, toutes les animations sont coupées proprement.

Animation	Déclencheur	Effet
wordIn	Chargement landing page	Mot qui monte avec légère rotation (0.6s)
shimmer	Mot clé en en-tête	Dégradé or qui défile en boucle (3.4s)
likePop	Clic sur un like	Scale 1 → 1.35 → 0.9 → 1 (rebond)
sparkle	Animation like	Particules qui s'éloignent du centre
toast-in/out	Notification UI	Glissement depuis la droite de l'écran
bob	Téléphone démo landing	Flottement doux en boucle (6s)

6.1.6 Principes UX

Mobile-first. Toute l'interface est conçue en partant du format mobile puis adaptée aux grands écrans via Tailwind. La navigation principale se déplace en barre basse sur mobile et en sidebar gauche sur desktop.

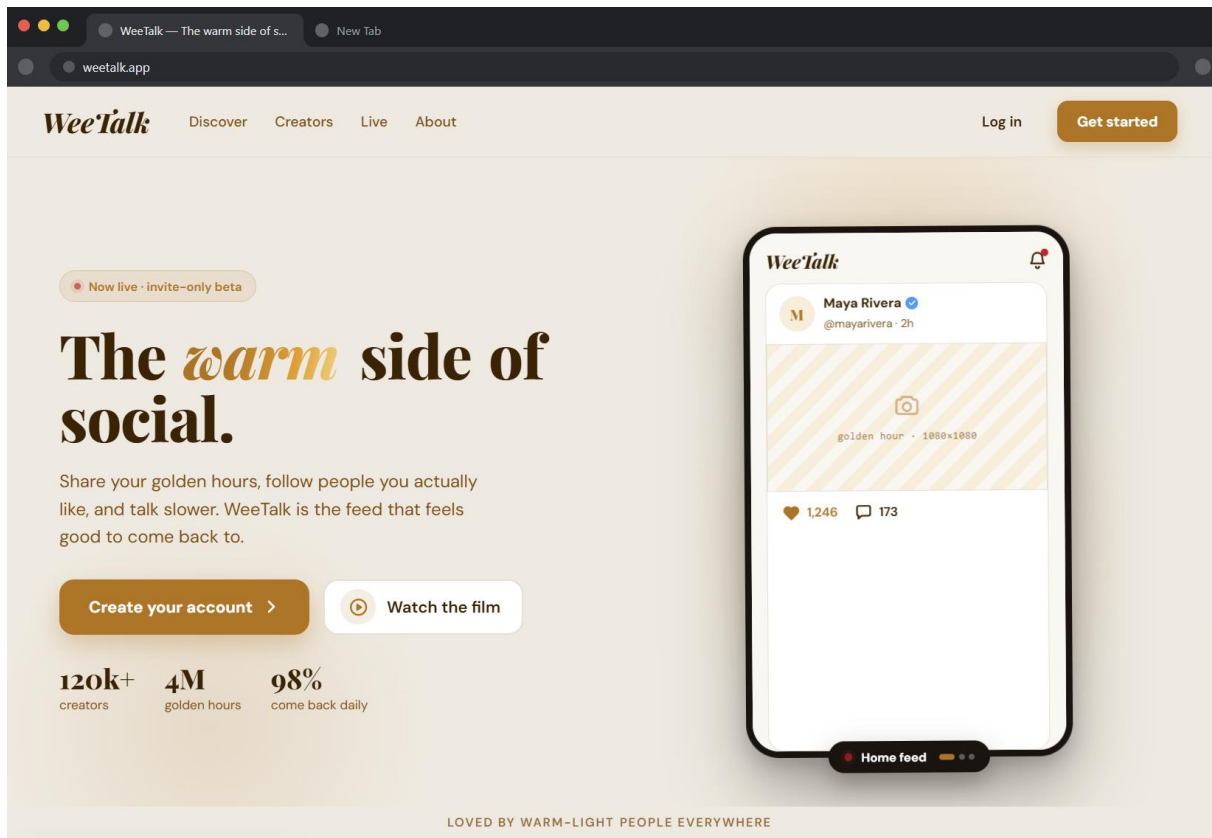
Accessibilité. Les composants interactifs (modales, menus déroulants, dialogs) sont tous construits avec Radix UI, qui gère nativement la navigation clavier, les rôles ARIA et le piège de focus dans les modales.

Retour visuel immédiat. Chaque action utilisateur est confirmée — un like déclenche une animation, un message envoyé apparaît instantanément, une erreur s'affiche en toast. L'utilisateur ne reste jamais dans l'incertitude sur l'état de son action.

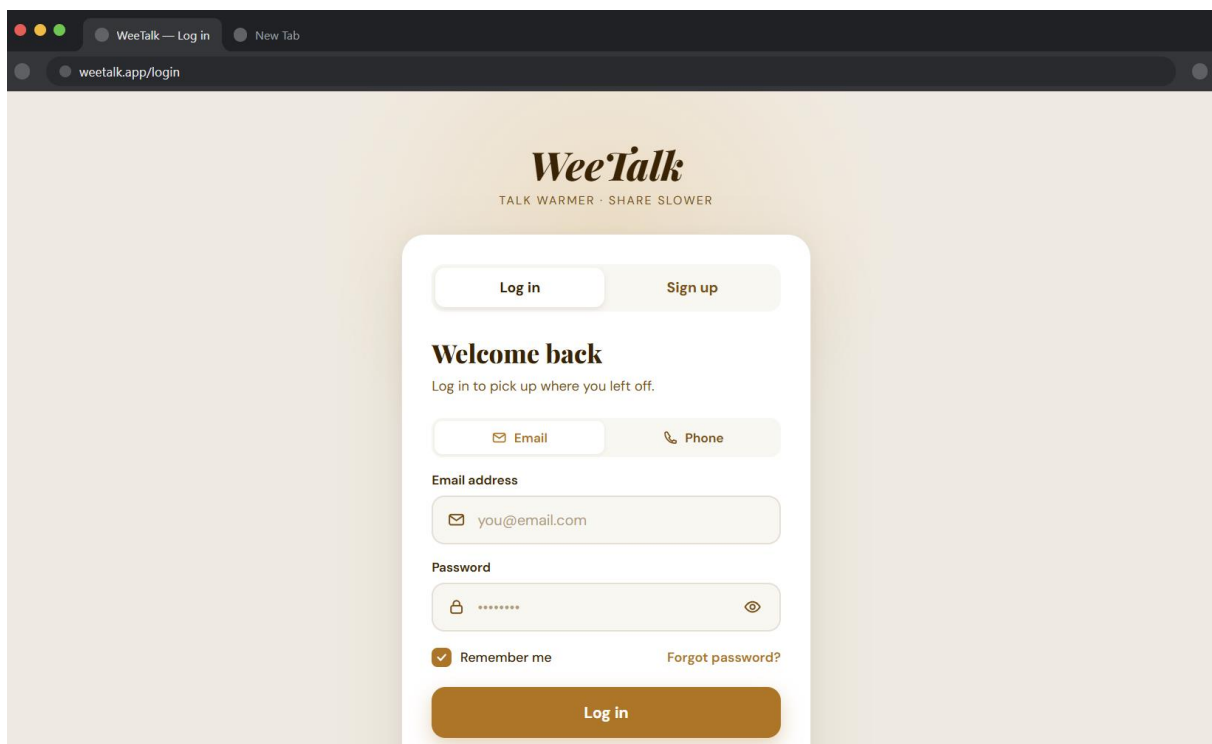
Temps réel sans friction. Les messages et les notifications arrivent sans que l'utilisateur ait à faire quoi que ce soit — pas de bouton "rafraîchir", pas de polling visible. Socket.io gère ça en arrière-plan.

Internationalisation. Toute la chaîne de caractères de l'interface est externalisée dans des fichiers JSON (fr.json / en.json). L'utilisateur change de langue depuis les paramètres et l'interface bascule immédiatement, sans rechargement.

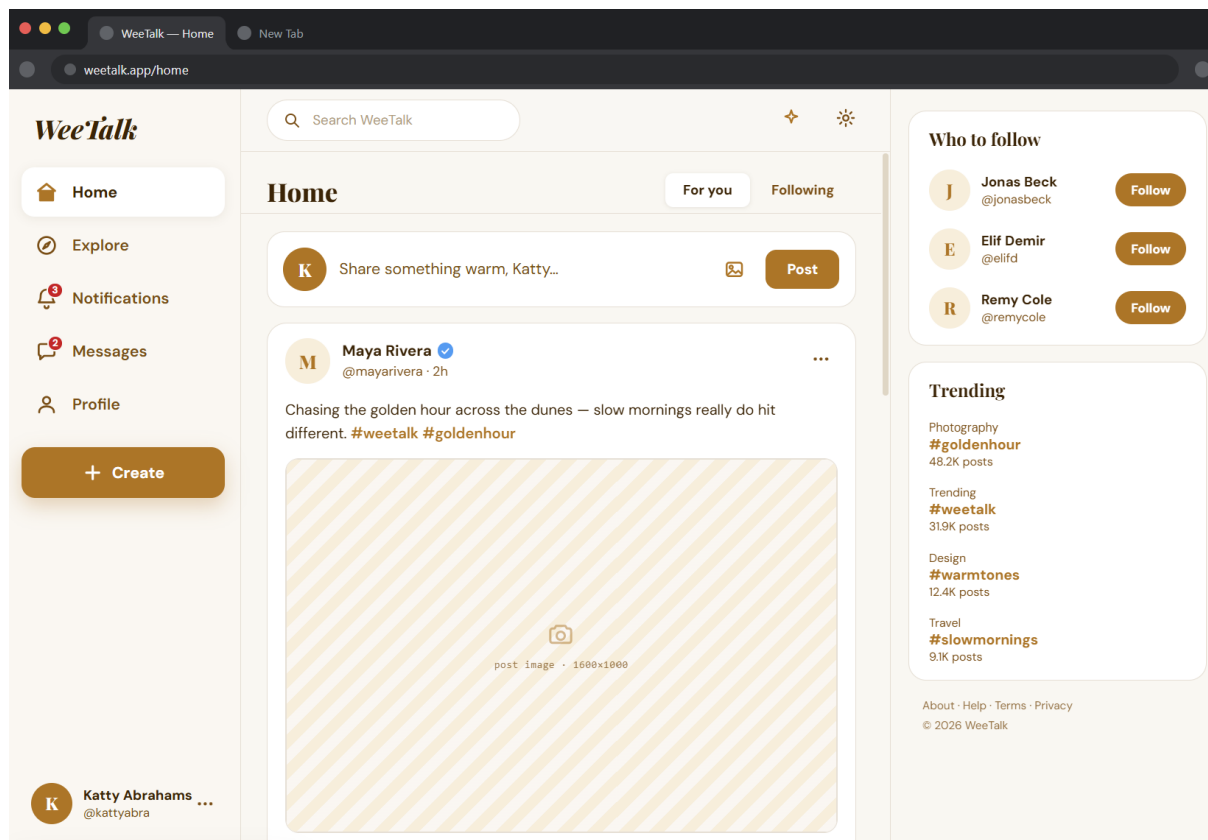
6.2 Maquettes ou wireframes des écrans principaux



Maquettes « Welcome-page » Front-end v1



Maquettes « Login page » Front-end v1



Maquettes « Home page » Front-end v1

6.3 Navigation et parcours utilisateur

La navigation dans WeTalk suit une logique simple et linéaire pour faciliter au mieux l'expérience des utilisateurs.

Premièrement, l'utilisateur ne peut accéder à l'application qu'après s'être authentifié, autrement, celui-ci n'a accès à aucune des fonctionnalités présentes sur le site (accès à la page du fil d'actualité, du profil ou des notifications). C'est la passerelle Nginx qui joue le premier rôle de filtre en redirigeant toutes les requêtes vers le service concerné (API Gateway)

6.3.1 L'authentification

Le point d'entrée de l'application est la page de bienvenue, suivi de la page de connexion :

- **L'utilisateur est déjà inscrit** : il saisit ses identifiants, le formulaire envoie une requête à l'auth-service via Nginx. En cas de succès, un cookie `wetalk_session` est posé côté client et l'utilisateur est redirigé vers son fil et peut commencer à utiliser l'application avec les fonctionnalités qui lui sont allouées.

- **Nouvel utilisateur** : il passe par la page d'inscription, qui crée son compte via l'auth-service puis l'user-service, avant de le rediriger automatiquement vers le son fil.

En cas d'échec (mauvais identifiants, compte inexistant), un message d'erreur est affiché pour prévenir l'utilisateur.

6.3.2 Navigation principale

Lorsque l'utilisateur est connecté, sur la partie gauche de l'application, l'utilisateur a accès aux sections principales qui sont :

- Le fil d'actualité (l'accueil par défaut) qui sollicite le post-service afin de consulter les publications des autres utilisateurs.
- La partie « Explorer » qui recense les derniers utilisateurs qui se sont inscrits sur l'application, et les tags en tendances.
- La partie « Notifications » qui sollicite le notification-service et qui liste les événements reçus (like, mentions etc.)
- La partie « Messages » qui sollicite le message-service qui permet de communiquer entre utilisateurs.
- La partie « Profil » qui permet de consulter et éditer son profil en sollicitant le message-service

6.3.3 Parcours pour publier

La fonctionnalité clé d'un réseau social est le fait de pouvoir poster du contenu à travers son profil pour communiquer avec les autres utilisateurs et donner une première impression, c'est pour cette raison que nous avons dédié un service sur le fait de pouvoir poster sur l'application. La création d'un post suit le chemin qui est le suivant :

- 1. L'utilisateur remplit le formulaire du post (message, gif, image..)
- 2. La requête est récupérée par le post-service via Nginx
- 3. Le post-service va vérifier le statut de l'utilisateur en sollicitant le « user-service » (si il est banni par exemple)

- 4. Si tout est okay ➔ la publication est enregistrée dans la BDD (MongoDB)
- 5. Le post-service va déclencher une notification via le notification-service
- 6. L'utilisateur pourra voir son post visible sur le fil d'actualité.

6.3.4 Déconnexion

La déconnexion supprime le cookie « wetalk_session » côté client + invalider sa session côté « auth-service ». L'utilisateur qui s'est déconnecté est redirigé vers la page de connexion. S'il y'a une tentative d'accès à une route protégée sans cookie valide renvoie une erreur 401 et redirige vers la page de connexion.

7. Fonctionnalités principales

7.1 Authentification et gestion des utilisateurs

La gestion des utilisateurs est gérée par un service indépendant, appelé `user-service`, développé avec le framework `Express.js`. Ce service crée, met à jour, et récupère des utilisateurs de la base de données, et est utilisé par d'autres services, notamment le `auth-service`.

Composants techniques et Implémentation :

- **Architecture logicielle** : Structure MVC (Modèle-Vue-Contrôleur) sous `Express.js` avec isolation de la couche d'accès aux données (ORM) avec `Sequelize` pour l'accès à `Postgres` et des cas d'usage (routes).
- **Sécurisation des données** : Intégration de middlewares `Express` pour vérification d'authentification et la validation stricte des schémas d'entrée (via `Zod`), ainsi que la vérification de rôle et le `rate limiter`.
- **Stratégie d'Authentification** : Utilisation d'un flux `stateless` basé sur des paires de jetons `JWT` (*access_token* à courte durée de vie et *refresh_token* sécurisé en cookie `HttpOnly`).

Spécifications des Routes API :

- `GET /api/users?search=query` : Rechercher des utilisateurs (avec champ de recherche).
- `GET /api/users/:id_ou_username` : Récupérer les informations publiques d'un profil.
- `GET /api/users/:id_ou_username/status` : Récupérer les informations publiques d'un profil.
- `GET /api/users/latest` : Retourne les 50 derniers utilisateurs inscrits.
- `GET /api/users/me` : Retourne les informations de l'utilisateur actuellement connecté.
- `PATCH /api/users/me` : Met à jour les informations de l'utilisateur actuellement connecté.
- `DELETE /api/users/me` : Supprime l'utilisateur actuellement connecté.
- `GET /api/users/:id/following` : Retourne la liste des abonnements d'un utilisateur.
- `GET /api/users/:id/following/ids` : Retourne la liste des identifiants des abonnements d'un utilisateur.
- `GET /api/users/:id/followers` : Retourne la liste des abonnés d'un utilisateur.
- `POST /api/users/:id/follow` : Abonne l'utilisateur connecté à un autre utilisateur.

- DELETE /api/users/:id/follow : Désabonne l'utilisateur connecté d'un autre utilisateur.
- POST /api/users/:id/ban : Banne un utilisateur.
- DELETE /api/users/:id/ban : Dé-banne un utilisateur.
- POST /api/users/:id/suspend : Suspend un utilisateur.
- DELETE /api/users/:id/suspend : Supprime la suspension d'un utilisateur.

7.2 Gestion des publications (posts)

Le service de gestion des publications (post-service) est développé avec Express.js et s'appuie sur une base MongoDB (via l'ODM Mongoose) pour stocker les contenus et leurs interactions.

Toutes ses routes sont sécurisées et exigent un jeton d'authentification valide. Avant chaque publication, le service s'assure auprès du user-service que l'auteur n'est ni banni ni suspendu.

Les potentielles images ou vidéos sont envoyées au media-service, qui retourne un lien d'accès. En cas d'échec de la publication après le téléversement, une suppression de compensation est déclenchée pour effacer le fichier stocké et éviter les données orphelines.

À la création d'un post (limité à 280 caractères), le texte est analysé pour en extraire :

- Les tags (mots précédés de #, passés en minuscules, maximum 10 par post).
- Les mentions (pseudonymes précédés de @), validées auprès du user-service.

Une notification est immédiatement envoyée via le notification-service à l'utilisateur mentionné, tout comme lors d'un nouveau commentaire ou d'un "j'aime".

Par souci de confidentialité, l'API ne renvoie jamais la liste brute des profils au client, mais seulement le compteur total et un indicateur booléen signalant si l'utilisateur connecté a aimé le contenu.

Pour composer le fil d'actualité, le service récupère la liste des abonnements du lecteur auprès du user-service, regroupe les publications correspondantes et les trie par ordre chronologique décroissant. Si un auteur est banni, ses publications sont masquées dynamiquement à la lecture, rendant la mesure réversible sans altérer la base de données.

Spécifications des Routes API :

- POST /api/posts : Créer une publication (texte + média optionnel).
- GET /api/posts : Lister les publications (filtres par auteur ou par tag disponibles).

- GET /api/posts/feed : Récupérer le fil d'actualité personnalisé (abonnements + ses propres posts).
- GET /api/posts/liked : Lister les publications aimées par un utilisateur.
- GET /api/posts/count : Compter les publications (par auteur, ou total global pour les admins).
- GET /api/posts/:id : Récupérer une publication spécifique.
- DELETE /api/posts/:id : Supprimer une publication (auteur ou modérateur/admin).
- POST /api/posts/:id/like : Aimer une publication.
- DELETE /api/posts/:id/like : Retirer son « j'aime ».
- GET /api/posts/:id/likes : Lister les utilisateurs ayant aimé une publication (paginé).
- POST /api/posts/:id/report : Signaler une publication pour modération.
- POST /api/posts/:id/comments : Commenter une publication.
- GET /api/posts/:id/comments : Lister les commentaires d'une publication.
- GET /api/comments : Lister les commentaires rédigés par un utilisateur spécifique.
- PATCH /api/comments/:id : Modifier un commentaire.
- DELETE /api/comments/:id : Supprimer un commentaire.
- POST /api/comments/:id/like : Aimer un commentaire.
- DELETE /api/comments/:id/like : Ne plus aimer un commentaire.
- GET /api/comments/:id/likes : Lister les utilisateurs ayant aimé un commentaire.
- GET /api/tags/trending : Récupérer les tags les plus populaires.
- GET /api/admin/... : Accéder aux fonctionnalités de modération des contenus

7.3 Système de notifications

Le système de notifications est géré par un service indépendant, appelé notification-service, développé avec le framework Express.js.

Ce service crée, liste, et marque comme lues des notifications en base de données, et assure la diffusion en temps réel via WebSocket (Socket.IO) vers les utilisateurs connectés.

Il est sollicité par d'autres services (post-service, user-service, message-service) via des appels internes.

Composants techniques et Implémentation :

- **Architecture logicielle** : Structure MVC (Modèle-Vue-Contrôleur) sous Express.js avec isolation de la couche d'accès aux données (ODM) via Mongoose pour l'accès à MongoDB et des cas d'usage (routes). Socket.IO est intégré pour permettre une diffusion en temps réel des notifications aux utilisateurs connectés.
- **Sécurisation des données** : Intégration de middlewares Express pour vérification d'authentification JWT et validation stricte des schémas d'entrée (via Zod), ainsi que du rate limiting différencié (300 req/15min en lecture, 60 req/15min en écriture). La route de création interne est uniquement réservée aux services internes et bloquée par la passerelle NGINX.
- **Notifications en temps réel** : Utilisation de Socket.IO avec authentification JWT lors du handshake. Chaque utilisateur rejoint une room dédiée `user:{userId}` et reçoit instantanément les événements le concernant (`notification:new`) et qui sont déclenchés par les actions des autres utilisateurs (like, mention, commentaire etc)

Spécifications des Routes API :

- GET `/api/notifications` : Liste paginée (cursor-based) des notifications de l'utilisateur connecté (20 par défaut, 50 max).
- GET `/api/notifications/unread` : Retourne le nombre de notifications non lues de l'utilisateur connecté.
- PATCH `/api/notifications/:id/read` : Marque une notification comme lue (vérification de propriété).
- POST `/api/notifications/internal` : Crée une notification et déclenche l'émission temps réel (réservé aux services internes, inaccessible depuis l'extérieur). Types de notifications supportés : like, comment, follow, mention.

7.4 Système de messagerie privée

Comme la plupart des services, le système de messagerie privée (`message-service`) utilise Express.js dans un format MVC, et une connexion à MongoDB via le package `mongoose`. Son objectif est de permettre aux utilisateurs de s'écrire des messages privés, en utilisant un WebSocket (via `Socket.io`), afin de maintenir une communication persistante entre le serveur et l'utilisateur pour transmettre les messages en temps réel. Il intègre une logique de communication inter-services pour valider l'existence des utilisateurs auprès du `user-service`.

Composants techniques et Implémentation :

- **Gestion des Flux de Données** : Implémentation de contrôleurs Express optimisés pour la lecture paginée (pagination par curseur) afin de garantir des temps de réponse bas lors du chargement des historiques de discussion.
- **Sécurité et Isolation** : Intégration d'un middleware d'autorisation capable de valider le JWT de l'expéditeur et de vérifier les droits d'accès à une conversation spécifique.
- **Communication Inter-services** : Mise en place d'appels HTTP asynchrones (via Axios) vers le user-service pour la résolution et la validation des identifiants utilisateurs (userId) avant toute création de message.

Spécifications des Routes API :

- GET /api/messages : Retourne la liste des conversations de l'utilisateur connecté.
- GET /api/messages/:id : Retourne une conversations entre l'utilisateur connecté et un autre utilisateur.
- POST /api/messages/:id : Envoi un message entre l'utilisateur connecté et un autre utilisateur.
- PATCH /api/messages/:id/read : Indique les messages d'une conversation entre l'utilisateur connecté et un autre utilisateur comme lu.
- DELETE /api/messages/:id : Supprime une conversation entre l'utilisateur connecté et un autre utilisateur.

7.5 Passerelle API et routage (Gateway Nginx)

L'ensemble de l'architecture microservices est unifié derrière une passerelle unique gérée par Nginx. En centralisant toutes les connexions sur le port 80, cette structure cache l'organisation interne du réseau et protège les serveurs. L'application utilisateur (le frontend) n'a ainsi besoin de connaître qu'une seule adresse globale, qui se charge de distribuer automatiquement chaque demande vers le bon service.

Pour la sécurité et la flexibilité de l'infrastructure, les différents services du projet sont regroupés au sein du réseau interne totalement isolé de Docker. Cette restriction permet de masquer les ports spécifiques de chaque service et de bloquer les attaques directes depuis l'extérieur, en permettant aux micro-services de communiquer entre eux de manière sécurisée.

L'aiguillage des demandes se fait grâce à l'analyse de l'adresse web saisie par l'utilisateur. Dès qu'une demande commence par /api/, la passerelle le retire automatiquement pour nettoyer l'adresse avant de l'envoyer au service concerné ; par exemple, une demande pour /api/users/ est nettoyée puis transférée directement au service des utilisateurs.

Pour les fonctionnalités en temps réel comme le chat ou les alertes instantanées, la passerelle est capable de transformer une connexion web classique en un canal de communication via websocket.

Afin que chaque service comprenne d'où vient la demande, la passerelle ajoute automatiquement des étiquettes d'identification à chaque message, contenant l'adresse internet d'origine de l'utilisateur et son protocole de sécurité. De plus, le système s'accorde un délai de 90 secondes pour finaliser les actions les plus lourdes sans couper la communication avec l'utilisateur.

La sécurité du réseau est renforcée en séparant strictement les accès publics des communications internes entre services. Les liaisons privées qui permettent aux services de se parler entre eux sont totalement bloquées pour les utilisateurs externes. Aussi, une limite de 60 Mo est imposée pour bloquer les fichiers trop lourds comme les vidéos et éviter les pannes.

Spécifications des Routes API (forward) :

- `/api/auth/` : authentification (inscription, connexion, rafraîchissement de jeton, déconnexion).
- `/api/users/` : profils, recherche et graphe social (abonnements/abonnés).
- `/api/posts` et `/api/posts/` : liste, création et gestion des publications.
- `/api/comments/` : commentaires et réponses.
- `/api/tags/` : tags et sujets en tendance.
- `/api/admin/` : modération (contenus signalés).
- `/api/notifications/` : notifications (REST).
- `/api/socket.io/notifications/` : flux temps réel des notifications (WebSocket).
- `/api/messages` et `/api/messages/` : conversations et messages privés (REST).
- `/api/socket.io/messages/` : flux temps réel de la messagerie (WebSocket).
- `/api/media/` : upload et distribution des médias (images, vidéos).
- `/api-docs/auth/`, `/api-docs/users/`, `/api-docs/posts/`, `/api-docs/notifications/`, `/api-docs/messages/`, `/api-docs/media/` : documentation Swagger de chaque service.
- `/api-docs/` : documentation Swagger par défaut.
- `/` : web : application front-end Next.js (toutes les routes non préfixées par `/api`).

8. Perspectives et axes d'amélioration

8.1 Évolutions fonctionnelles envisagées

Afin d'enrichir l'interactivité de la plateforme et de s'aligner sur les standards des réseaux sociaux modernes, plusieurs chantiers fonctionnels sont envisagés pour les prochaines itérations :

- **Refonte du système de publication multi-contenus** : Permettre aux utilisateurs d'attacher plusieurs images à un même message (avec un seuil maximal prédéfini) et étendre les capacités des commentaires en y intégrant la possibilité d'ajouter des fichiers multimédias (images et GIFs).
- **Mécanismes d'engagement et de viralité** : Implémentation d'une fonctionnalité native de partage de publication (*retweet/repost*) pour maximiser la circulation du contenu au sein du réseau.
- **Système de certification des comptes** : Introduction d'un processus de vérification de l'identité des profils (badge de certification) afin de valoriser les créateurs de contenu officiels et limiter l'usurpation d'identité.
- **Géolocalisation optionnelle** : Offrir la possibilité aux utilisateurs d'associer une position géographique à leurs publications ou à leur profil, favorisant ainsi les interactions locales.

8.2 Améliorations techniques (scalabilité, monitoring, CI/CD...)

L'augmentation du volume d'utilisateurs et de médias échangés nécessite de consolider l'architecture technique sur les aspects de sécurité et d'infrastructure :

- **Renforcement de la sécurité à l'inscription (Onboarding)** : Intégration d'un module de vérification d'adresse email obligatoire lors de la création de compte, afin de valider l'authenticité des utilisateurs et de lutter contre la création massive de comptes automatisés (bots).
- **Gestion et optimisation du stockage des médias** : Face à l'introduction du multi-image et des GIFs, optimisation des pipelines d'upload avec compression automatique à la volée côté serveur et distribution via un réseau de diffusion de contenu (CDN).
- **Orchestration et haute disponibilité des conteneurs (Kubernetes)** : Transitionner de l'architecture Docker standard vers un orchestrateur de conteneurs afin de simplifier le déploiement à grande échelle, d'automatiser le self-healing des services et de gérer efficacement la montée en charge. L'utilisation d'une solution légère comme **k3s** sera privilégiée pour les environnements de staging ou les ressources limitées, tandis qu'une bascule vers une distribution **k8s** complète sera envisagée pour la production hautement disponible.

8.3 Pistes d'optimisation de l'expérience utilisateur (UX)

L'expérience utilisateur doit être fluidifiée pour garantir une navigation agréable et offrir un environnement hautement personnalisable et sécurisant :

- **Organisation thématique du profil :** Création d'un onglet dédié aux « Médias » sur le profil des utilisateurs, permettant de centraliser et de parcourir rapidement l'ensemble des images et vidéos publiées par un compte, sans devoir faire défiler tout son flux de texte.
- **Intégration d'un sélecteur de GIFs natif :** Intégration d'une API tierce (type Giphy ou Tenor) directement dans l'interface de composition des messages pour permettre la recherche et l'insertion de GIFs en un clic.
- **Outils de modération et de contrôle de la vie privée :**
 - *Côté utilisateur :* Implémentation de fonctionnalités de masquage de contenu et de blocage de comptes pour permettre à chacun de modérer son propre fil d'actualité.
 - *Côté plateforme :* Mise en place d'un système de signalement des comportements ou contenus inappropriés, envoyant les rapports vers un back-office de modération dédié.

Conclusion

Pour conclure, WeTalk est désormais un réseau social développé qui correspond à nos attentes techniques, c'est-à-dire une architecture microservices complète et conteneurisée. L'ensemble des services sont opérationnels et communiquent au sein d'un environnement Docker, avec Nginx comme unique point d'entrée externe (API Gateway)

Le résultat est une base solide et modulable, nous soulignons également plusieurs axes d'amélioration qui restent ouverts tel que le passage à un orchestrateur comme Kubernetes, renforcer la sécurité en passant à l'Oauth 2.0 en passant par des sites tiers mais également ajouter un onglet « Médias » sur le profil utilisateur pour afficher ses images / vidéos publiées.

WeTalk démontre qu'il est possible, avec des outils accessibles et bien appliqués, de construire un système distribué robuste qui correspond aux attentes d'un véritable réseau social.